

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Software Engineering

Extended-Course Project Report

**Mini Software Engineering Project -
Smart Event Reminder System
(SERS)**

Class A01 – Group CNPM_TN1 – Semester 252

Advisor(s): Nguyễn Thành Công

Student(s): Nguyễn Hữu Thịnh 2313292

Nguyễn Chí Thanh 2313078

Nguyễn Việt Hoàng 2311066

Lê Đức Tài 2312995

Mai Xuân Nhựt 2312549

HO CHI MINH CITY, MAY 2026

Contents

1	Requirement Engeneering	1
1.1	Stakeholder Identification	1
1.2	Functional Requirement(FRs)	1
1.3	Non-Functional Requirements (NFRs)	2
1.4	User Stories & Acceptance Criteria	2
1.5	MoSCoW Prioritization Framework	4
1.6	Requirements Traceability Matrix (RTM)	4
2	Process Model and Planning	5
3	Architectural Design	9
4	Implementation & Code Quality	9
5	Software Testing and Quality Assurance	13
5.1	Test Plan	14
5.2	Test Cases	16
5.3	Quality Assurance Discussion	17
6	Reflection and Retrospective	17

1 Requirement Engineering

1.1 Stakeholder Identification

The initial phase involves identifying all entities that interact with, benefit from, or govern the software application.

- **User:** The primary human consumer of the application who interfaces with the graphical user interface to manage calendar events.
- **System Administrator:** An operational overseer equipped with elevated privileges, responsible for managing accounts, monitoring system health, and resolving data conflicts.
- **Notification Service:** An internal programmatic actor or external third-party integration (like an SMTP server or SMS gateway) that routes and delivers asynchronous reminders.
- **Project Engineering Team:** The five-member technical unit responsible for designing, implementing, verifying, and maintaining the system architecture.
- **Instructor:** The governing authority responsible for evaluating the fidelity of the software engineering process and documentation.

1.2 Functional Requirement(FRs)

Functional requirements define the explicit behaviors and data processing operations the system must execute

- **FR-01: Event Lifecycle Management:** The system must enable an authenticated user to create, read, update, and delete (CRUD) event entries, enforcing a valid future date and time parameter.
- **FR-02: Metadata Association:** The system must allow users to append optional metadata, specifically expanded textual descriptions and geographical location data.
- **FR-03: Alert Schedule Calculation:** The system must calculate a specific notification trigger timestamp based on the user-defined reminder interval subtracted from the primary event timestamp.
- **FR-04: Asynchronous Alert Execution:** The system must asynchronously dispatch the reminder via the Notification Service at the calculated trigger timestamp without blocking the main application thread.



- **FR-05: User Authentication Protocol:** The system must authenticate users via secure credentials to ensure event data is strongly siloed.
- **FR-06: Event Sharing Interface:** The system should provide a mechanism allowing a user to share event details or digitally invite other registered users.

1.3 Non-Functional Requirements (NFRs)

Non-functional requirements act as systemic constraints, dictating how the system must behave under various operational loads

- **NFR-01: Performance and Latency:** The graphical user interface must respond to synchronous user inputs (e.g., saving a new event) in under 1,000 milliseconds to prevent user frustration.
- **NFR-02: System Scalability:** The asynchronous notification engine must be linearly scalable, capable of processing up to 50,000 asynchronous notifications per day without degrading the primary web server's performance.
- **NFR-03: Reliability and Availability:** The application infrastructure must maintain an operational uptime of 99.9

1.4 User Stories & Acceptance Criteria

User stories utilize a standard human-centric narrative format: “As a [type of user], I want [objective] so that [benefit]”. Each story is accompanied by explicit Acceptance Criteria to define the conditions for completion.

- **US-01: Create Event**
 - **User Story:** As a user, I want to create a new event with a title, date, and time, so that I can maintain an accurate digital record of my upcoming appointments.
 - **Acceptance Criteria:**
 1. The input form cryptographically and logically validates the title, date, and time.
 2. The event object successfully persists to the database.
 3. The graphical interface displays a success confirmation modal.
- **US-02: Schedule Reminder**

- **User Story:** As a user, I want to set a specific reminder time interval prior to an event, so that I am alerted automatically before the occurrence begins.
- **Acceptance Criteria:**
 1. The system interface accepts numerical time intervals.
 2. The backend mathematical engine calculates the exact trigger time.
 3. The background scheduler successfully registers the task in the message broker queue.
- **US-03: Edit Event**
 - **User Story:** As a user, I want to edit an existing event's specific details, so that my schedule remains accurate if locations or times change unexpectedly.
 - **Acceptance Criteria:**
 1. The user can retrieve and populate an existing event into the input form.
 2. Modifications overwrite the existing database record.
 3. Any previously scheduled reminder triggers associated with the old time are successfully revoked and replaced.
- **US-04: Delete Event**
 - **User Story:** As a user, I want to delete a canceled event entirely, so that my application dashboard remains uncluttered and accurate.
 - **Acceptance Criteria:**
 1. The deletion command removes the event from the client view.
 2. The corresponding database record is permanently dropped.
 3. All pending notification tasks associated with the event ID are aborted in the background queue.
- **US-05: View System Logs**
 - **User Story:** As an administrator, I want to view system logs for failed notification deliveries, so that I can troubleshoot infrastructure or SMTP delivery issues proactively.
 - **Acceptance Criteria:**

1. The secure administrator dashboard queries and displays error logs.
2. The log entries explicitly indicate the failed event ID, the target user, and the failure timestamp.

1.5 MoSCoW Prioritization Framework

To prevent scope creep and ensure the delivery of a Minimum Viable Product (MVP), the team utilizes the MoSCoW prioritization framework.

Must Have: Features vital to the system's existence. This includes basic event creation, event deletion, logical reminder scheduling, and the asynchronous dispatch of the notification. Without these, SERS fails to function as a reminder application.

Should Have: Important features that add significant value but are not strictly existential. This includes event editing, recurring event configurations, and the ability to set multiple reminder intervals (e.g., an alert one hour prior, and another ten minutes prior).

Could Have: Desirable enhancements that will only be addressed if sprint capacity permits. This encompasses the optional functionality of inviting other users, sharing events via direct email links, and automated workflow suggestions based on calendar availability.

Won't Have (This Iteration): Complex functionalities explicitly deferred to future releases. This includes integration with third-party proprietary calendars (e.g., Google Calendar APIs), complex geographical routing for travel time alerts, and AI-driven predictive scheduling.

1.6 Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) mathematically proves that every functional requirement is tethered to a specific user story, which in turn maps to an architectural component and a definitive test case. This linkage guarantees that:

- No orphaned code is written without a corresponding requirement.
- No requirement is left untested.
- Every architectural component has a clear functional justification.

Functional Req.	Correlated User Story	Primary Architectural Component	Assigned Test ID
FR-01 (Event Lifecycle)	US-01, US-03, US-04	EventController, EventModel	T1, T2
FR-02 (Metadata)	US-01	EventModel, MetadataField	T1
FR-03 (Alert Calculation)	US-02	SchedulerService, TaskQueue	T3, T4
FR-04 (Async Dispatch)	US-02	NotificationWorker, SMTP Client	T3, T5
FR-05 (Authentication)	US-05	AuthController, UserModel	T6
FR-06 (Event Sharing)	US-05	SharingService, InviteModel	T7

Table 1.1: Requirements Traceability Matrix for SERS Project

2 Process Model and Planning

- **Process Model Selection**

For the Smart Event Reminder System (SERS), the team adopts the **Agile Scrum** model.

Scrum is selected because the project has a short duration (approximately two weeks), involves evolving requirements, and requires continuous integration between multiple components such as the user interface, backend logic, and asynchronous notification system. Compared to the Waterfall model, Scrum allows the team to quickly adapt to unexpected issues (e.g., scheduling errors or integration problems). Compared to the In-

cremental model, Scrum provides more structured collaboration through regular meetings and feedback loops.

Therefore, Scrum is the most suitable model for ensuring flexibility, collaboration, and timely delivery.

- **Product Backlog**

The Product Backlog contains all features required for the system, prioritized based on importance:

Table 2.1: Product Backlog

ID	Backlog Item	Priority
PB1	Create event (title, date, time)	Must
PB2	Edit event	Must
PB3	Delete event	Must
PB4	Schedule reminder	Must
PB5	Send notification	Should
PB6	User authentication	Should
PB7	Share event	Could

- **Effort Estimation (Planning Poker)**

The team uses Planning Poker with Fibonacci scale (0, 1, 2, 3, 5, 8, 13, 21) to estimate effort:

Table 2.2: Story Points Estimates

Backlog Item	Story Points
Database schema	3
CRUD API	8
Basic UI	10
Schedule reminder	8
Send notification	5
Authentication	8
Share event	3

- **Sprint Planning**

The project is divided into two sprints (1 week each), ensuring incremental delivery.

Sprint 1: Core System Setup (21 Story Points) The team commits to the following backlog items:

- Initialize database schema (3 points)
- Implement CRUD API for events (8 points)
- Develop basic UI dashboard (10 points)

Total: 21 story points

This sprint focuses on delivering a functional core system (Minimum Viable Product – MVP), allowing users to create, view, edit, and delete events.

—

Sprint 2: Advanced Features (Remaining Story Points) The second sprint focuses on extending system functionality:

- Reminder scheduling (8 points)
- Notification system (5 points)
- User authentication (8 points)
- Event sharing (3 points, optional)

This sprint enhances the system by adding automation, security, and additional user features.

—

This two-sprint structure ensures that a working system is delivered early (Sprint 1), while advanced capabilities are incrementally added in Sprint 2.

• Team Task Distribution

To support parallel development, tasks are distributed among five team members:

- Member A: Backend (CRUD API, database)
- Member B: Frontend (UI dashboard)
- Member C: Asynchronous system (reminder & notification)
- Member D: Testing and QA
- Member E: Documentation and integration

- **Sprint Day (Simulated 10-Day Cycle)**

Table 2.3: Sprint Day (Simulated 10-Day Cycle)

Day	Remaining Story Points	Progress Narrative
Day 1	21 Points	The sprint begins. All parallel workstreams initiate development on their assigned components.
Day 3	18 Points	The database schema architecture is finalized and committed. Points are burned down successfully.
Day 6	10 Points	The RESTful API backend logic is completed. The team maintains pace with the ideal burndown trajectory.
Day 8	8 Points	A slight plateau occurs. The frontend integration encounters cross-origin resource sharing (CORS) errors, delaying the dashboard completion.
Day 10	0 Points	Integration issues are resolved. The dashboard is completed, and all sprint objectives are achieved prior to the sprint review.

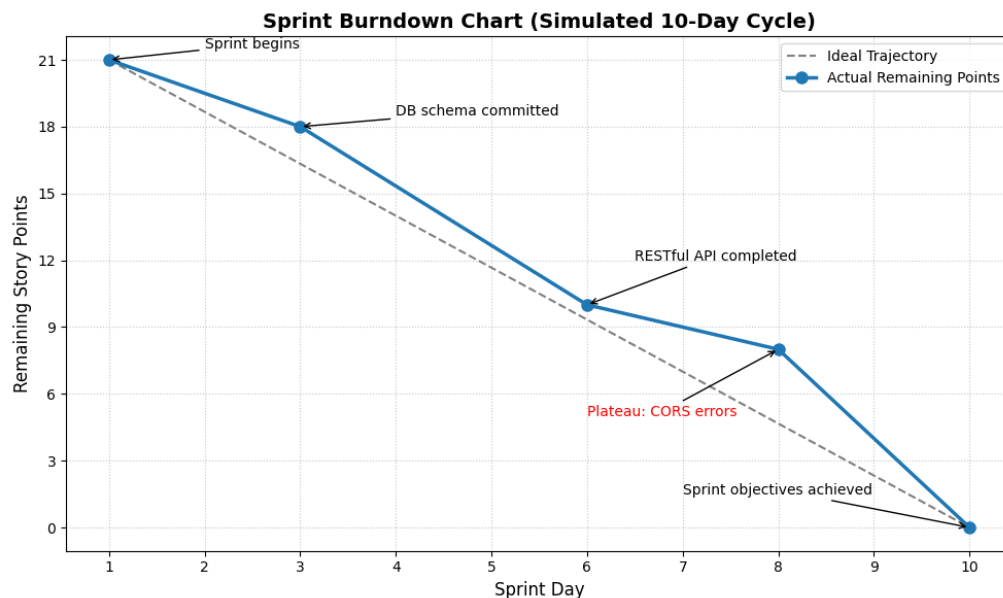


Figure 2.1: Burndown chart (Simulated 10-Day Cycle)

- **Summary**

Using Scrum enables the team to manage work effectively, adapt to technical challenges,

and deliver the system incrementally. The use of backlog prioritization, effort estimation, and sprint planning ensures that the project remains organized and achievable within the given timeframe.

3 Architectural Design

Translating the abstract UML models into a functioning codebase requires the selection of an overarching architectural pattern. The SERS application utilizes the Model-View-Controller (MVC) paradigm, heavily augmented by an asynchronous message-brokering layer. The MVC pattern enforces a rigorous separation of concerns. The Model layer encapsulates all business logic, data validation, and database interactions. The View layer is strictly confined to presentation logistics, rendering the user interface and accepting input. The Controller acts as the central orchestrator, receiving HTTP requests from the View, commanding the Model to update its state, and managing the application flow.

However, standard MVC is fundamentally synchronous; a Controller typically blocks the server thread until a task completes. For a reminder system, holding an HTTP connection open for three days until a scheduled event occurs is architecturally catastrophic. Therefore, the technology stack is designed to offload temporal logic. The backend Controller and Model are constructed using Python and the Flask framework. Data is persisted using an SQLite database (capable of scaling to PostgreSQL) mapped via SQLAlchemy. Crucially, the asynchronous layer utilizes Celery as the task queue manager and Redis as the in-memory message broker. When the Flask Controller receives a scheduling request, it pushes the task parameters into the Redis broker and immediately responds to the user. Independent Celery worker processes continuously monitor the Redis broker, executing the tasks only when their specific temporal conditions are met.

4 Implementation & Code Quality

The theoretical architecture is demonstrated in the following Python implementation snippet, which encapsulates the Controller logic and the Celery background worker.

```
1 import os
2 import datetime
3 from celery import Celery
4 from sqlalchemy import create_engine, Column, Integer, String, DateTime
```

```

5 from sqlalchemy.orm import declarative_base, sessionmaker
6
7 # Architecture Initialization: Message Broker and Database Config
8 DATABASE_URI = os.getenv('PROD_DATABASE_URL', 'sqlite:///sers_local.db')
9 REDIS_BROKER = os.getenv('CELERY_BROKER_URL', 'redis://localhost:6379/0')
10
11 app_celery = Celery('sers_async_worker', broker=REDIS_BROKER)
12 app_celery.conf.timezone = 'UTC'
13
14 engine = create_engine(DATABASE_URI)
15 Base = declarative_base()
16 Session = sessionmaker(bind=engine)
17
18 # Model Layer: Data encapsulation mapping to the UML Class Diagram
19 class EventEntity(Base):
20     __tablename__ = 'scheduled_events'
21     id = Column(Integer, primary_key=True)
22     title = Column(String(150), nullable=False)
23     event_timestamp = Column(DateTime, nullable=False)
24     target_email = Column(String(150), nullable=False)
25     async_task_id = Column(String(100)) # Stores Celery ID for revocation
26
27 Base.metadata.create_all(engine)
28
29 # Asynchronous Worker Layer: Executed completely independent of the web server
30 @app_celery.task(name='notification.dispatch', bind=True)
31 def dispatch_reminder_alert(self, entity_id, target_email, event_title):
32     """
33     Background worker simulating external SMTP connection.
34     This function blocks its own thread, but does not block the web server.
35     """
36     try:
37         alert_payload = f"Automated Reminder: '{event_title}' is approaching."
38
39         # System logging to verify successful delivery for administrative review
40         log_entry = f"[{datetime.datetime.utcnow().isoformat()}] SUCCESS -> {target_email} | Payload:
41         ↪ {alert_payload}\n"
42         with open("system_delivery_logs.txt", "a") as file_handle:
43             file_handle.write(log_entry)
44
45         return {"execution_status": "success", "entity_id": entity_id}
46     except Exception as runtime_error:
47         return {"execution_status": "failed", "error_trace": str(runtime_error)}
48
49 # Controller Layer: Orchestrates Model updates and Broker dispatch
50 class SersController:
51     def __init__(self):
52         self.db_session = Session()
53
54     def schedule_new_event(self, title, event_timestamp, target_email, offset_minutes):
55         """
56         Validates input, persists the Model, calculates temporal offsets,
57         and pushes the payload to the Redis broker.
58         """
59         # Primary logical validation
60         if not title or not target_email:
61             raise ValueError("Data Validation Failure: Title and Email are mandatory parameters.")
62
63         current_system_time = datetime.datetime.utcnow()
64         if event_timestamp <= current_system_time:

```

```

64         raise ValueError("Temporal Logic Failure: Scheduled events must occur in the future.")
65
66     # 1. Persist the Model state
67     new_event = EventEntity(
68         title=title,
69         event_timestamp=event_timestamp,
70         target_email=target_email
71     )
72     self.db_session.add(new_event)
73     self.db_session.commit()
74
75     # 2. Calculate precise temporal trigger offset
76     calculated_trigger = event_timestamp - datetime.timedelta(minutes=offset_minutes)
77     if calculated_trigger < current_system_time:
78         # Fallback mechanism if offset pushes trigger into the past
79         calculated_trigger = current_system_time + datetime.timedelta(seconds=5)
80
81     # 3. Dispatch the payload to the asynchronous broker
82     dispatched_task = dispatch_reminder_alert.apply_async(
83         args=[new_event.id, new_event.target_email, new_event.title],
84         eta=calculated_trigger
85     )
86
87     # 4. Update Model with task identifier to allow future revocation
88     new_event.async_task_id = dispatched_task.id
89     self.db_session.commit()
90
91     return new_event.id
92
93 def revoke_existing_event(self, entity_id):
94     """
95     Removes the Model entity and aggressively terminates the pending asynchronous task.
96     """
97     target_event = self.db_session.query(EventEntity).filter_by(id=entity_id).first()
98     if target_event:
99         if target_event.async_task_id:
100             # Intercept the message broker to prevent misfire of canceled events
101             app_celery.control.revoke(target_event.async_task_id, terminate=True)
102             self.db_session.delete(target_event)
103             self.db_session.commit()
104         return True
105     return False

```

The inspection process focuses on identifying Code Smells—structural indicators of poor design that, while not necessarily preventing the program from functioning, significantly increase technical debt and the probability of future catastrophic failures.

Table 4.1: Identified Code Smells and Refactoring Strategies

Identified Code Smell Category	Technical Description and Observation	Proposed Refactoring Strategy
Bloater: Long Method	The <code>schedule_new_event</code> method handles input validation, database instantiation, complex mathematical time calculations, and asynchronous message brokering simultaneously. It violates the Single Responsibility Principle by attempting to manage too many disparate layers of logic. ³	Extract Method: The mathematical calculation of the temporal offset and the actual dispatching to the Celery broker should be abstracted into a private helper function (e.g., <code>_dispatch_to_broker()</code>). This isolates the temporal logic, making the primary method vastly more readable and testable. ³
Object-Orientation Abuser: Primitive Obsession	Passing discrete primitive variables (<code>title</code> , <code>event_timestamp</code> , <code>target_email</code> , <code>offset_minutes</code>) linearly through multiple architectural layers heavily clutters the method signatures, resulting in Long Parameter Lists and code duplication. ⁴⁹	Introduce Parameter Object: The isolated primitives should be grouped into a single cohesive Data Transfer Object (DTO) or a formalized validation model (such as a Pydantic schema). Passing a single <code>EventPayload</code> object ensures strict type safety across boundaries. ³

Identified Code Smell Category	Technical Description and Observation	Proposed Refactoring Strategy
Concurrency Anti-Pattern: Unsafe Logging	Utilizing <code>open("system_delivery_logs.txt", "a")</code> directly within the asynchronous worker is highly dangerous. If multiple Celery workers attempt to append to the same text file simultaneously, it will result in file lock collision errors and lost data.	Implementation Refactor: Remove raw file I/O operations from background workers. Replace with a thread-safe Python logging module configuration or route logs through a secondary stream processing broker.

5 Software Testing and Quality Assurance

The purpose of testing in SERS is to verify that the most important user-facing and system-level behaviors work correctly before the report is finalized. Based on the requirements in Workstream 1 and the MVC plus asynchronous architecture described in Workstream 4, the team selected three key behaviors for validation: creating an event, calculating and scheduling a reminder, and revoking a reminder when an event is deleted. These behaviors directly support the core user stories US-01, US-02, and US-04, and they also cover the highest-risk logic in the current design.

The scope of testing is limited to the conceptual backend flow presented in the code sample: controller validation, event persistence, reminder time calculation, and interaction with the asynchronous task queue. Full end-to-end delivery through a live SMTP provider is outside the scope of this assignment because the project emphasizes process and documentation rather than production deployment. Therefore, notification delivery is validated through mocked or simulated execution results rather than external email transmission.

5.1 Test Plan

Table 5.1: Test Plan for Key SERS Functions

Test Item	Objective	Test Level	Method
Create event (US-01, FR-01)	Verify that valid event data is accepted, stored, and returned correctly by the controller layer.	System	Black-box
Schedule reminder (US-02, FR-03, FR-04)	Verify that the system computes the reminder trigger time correctly and registers the asynchronous task without blocking the main flow.	Unit / Integration	White-box
Delete event and revoke task (US-04, FR-01, FR-04)	Verify that deleting an event removes the database record and revokes the corresponding queued reminder task.	Integration / System	Black-box



5.2 Test Cases

Table 5.2: Representative Test Cases

Test ID	Description	Input	Expected Output	Test Type
T1	Create event with valid data	Title = “Architecture Sync”; Event time = future timestamp; Email = “admin@sers.local”; Offset = 15 minutes	The controller accepts the request, stores the event in the database, schedules a reminder task, and returns a valid event identifier.	Black-box
T2	Reject event with missing required title	Title = empty string; Event time = future timestamp; Email = “admin@sers.local”; Offset = 15 minutes	The system raises a validation error and no event record is committed to the database.	Black-box
T3	Verify reminder trigger calculation	Event time = 10:00 UTC; Offset = 15 minutes; Simulated current time = 09:40 UTC	The computed reminder trigger is 09:45 UTC, and the task is submitted to the queue with the correct execution time.	White-box
T4	Reject event scheduled in the past	Title = “Retrospective”; Event time = past timestamp; Email = “admin@sers.local”; Offset = 10 minutes	The controller raises a temporal validation error because events must be scheduled in the future.	White-box
T5	Delete event and revoke queued reminder	Invoke <code>revoke_existing_event(entity_id)</code> for an existing scheduled event	The event is deleted from the database and the associated Celery task ID is revoked so the reminder is not dis-	System

5.3 Quality Assurance Discussion

Quality assurance in this project is not limited to running test cases. It also includes checking whether the implementation remains consistent with the requirements, architecture, and code quality goals defined in the earlier workstreams. The team used the Requirements Traceability Matrix from Workstream 1 to ensure that each critical requirement is supported by at least one test case. For example, FR-01 is covered by T1, T2, and T5, while FR-03 and FR-04 are covered by T3 and T5. This traceability reduces the risk of implementing features that are not verified or testing behavior that is not tied to a requirement.

From a code quality perspective, Workstream 4 already identified several risks in the implementation, including a long controller method, primitive obsession in the parameter list, and unsafe file logging in an asynchronous worker. These findings are also QA concerns because they affect testability, maintainability, and operational reliability. For that reason, the team recommends refactoring the controller into smaller units, introducing a validated data transfer object for event input, and replacing direct file writes with the standard logging module or a centralized logging service. These changes would make automated testing easier and would reduce defects caused by concurrency or inconsistent data handling.

Although this report does not require a full executable test suite, the proposed automation strategy is straightforward. Unit tests can be implemented with `pytest` to validate controller logic and time calculations. Integration tests can use a temporary SQLite database and a mocked Celery interface to confirm that scheduling and revocation happen correctly. For time-dependent behavior, clock-mocking utilities such as `freezegun` can simulate future timestamps without waiting in real time, which is especially important for reminder systems.

6 Reflection and Retrospective

At the end of the project, the team conducted a short retrospective to evaluate both technical execution and collaboration. Because the project was divided into parallel workstreams, the discussion focused on coordination quality, handoff clarity, and how well shared artifacts such as requirements, diagrams, and code samples supported integration.

Table 6.1: Start / Stop / Continue Retrospective

Category	Team Reflection
Start	Begin each sprint with a more explicit interface agreement, especially for event data format, datetime handling, and reminder parameters. This would reduce rework between requirements, architecture, implementation, and testing tasks.
Stop	Stop making isolated assumptions about shared components without confirming them with the rest of the team. In particular, asynchronous behavior and validation rules must be agreed on early to avoid inconsistencies across sections of the report.
Continue	Continue using shared UML diagrams, backlog planning, and peer review. These practices helped the team stay aligned and made it easier to connect requirements, architecture, code quality, and testing in one coherent report.

Overall, the team learned that even a small reminder system becomes significantly more complex once asynchronous behavior is introduced. The project highlighted the importance of clear requirements, realistic scope control, and consistent communication between parallel workstreams. It also reinforced several practical software engineering lessons: estimate risk early, define shared interfaces before implementation, and treat testing as a design activity rather than a final checklist. These insights are directly applicable to future collaborative software projects.

References

- [1] John Smith and Jane Doe. “A Comprehensive Study of Machine Learning Algorithms”. In: *Journal of Artificial Intelligence Research* 15.3 (June 2020), pp. 123–145. DOI: [10.1234/jair.2020.12345](https://doi.org/10.1234/jair.2020.12345).
- [2] Michael Johnson. *Introduction to Software Engineering*. 3rd. New York, NY: Academic Press, 2019.
- [3] Alice Lee and Bob Chen. “Deep Learning for Image Recognition”. In: *Proceedings of the International Conference on Computer Vision*. Virtual Event, Oct. 2021, pp. 456–467.